

# Study C — Evidence Integrity Under a Hostile Executor: Changelog

Full revision history for the project’s primary paper — `report.md` through v20, renamed to `study-c-evidence-integrity.md` shortly after (see the dated note at the end of this file) — moved out of the main report in v10 to keep the primary document focused (an external review correctly noted that the inline revision history — roughly a third of the document by length at the time — made the report harder to read than the underlying research warranted). Nothing here is summarized or reworded from what was previously in the report; every entry below is unchanged from its original wording, including references to `report.md` by the name it had at the time each entry was written. The current file retains a short pointer plus a one-paragraph summary of the most recent round; this file is the complete record, oldest first.

## Changes from v1

An external review of the first draft identified two material errors and several framing issues, all addressed above rather than smoothed over:

1. **The falsifier assessment was incomplete.** v1 declared the central claim “not falsified” using a joint FA/FR criterion not present in the literal preregistered falsifier text. v2 states plainly that Gemini’s result triggers the narrow, literally-worded falsifier, and presents the joint-discrimination result as the separate, secondary finding it actually is (see “Assessment of the falsifier”).
2. **The 90% adaptive figure was mislabeled.** v1’s executive summary and adaptive section described DeepSeek’s execution false-acceptance rate as climbing to 90% at budget 8. The correct false-acceptance figure is 75% (15/20, still-false); 90% (18/20) is *total* acceptance, including 3 cases where the search converged onto the literal true result and are not false acceptances. v2 reports both numbers, separated, everywhere this appears.
3. **Confirmatory and exploratory results were not separated.** v2 labels the execution-claim adaptive analysis as exploratory and post hoc throughout, and does not treat it as resolving the underpowered preregistered DeepSeek-vs-reference paired comparison.
4. **P3 was described as having “solved” the problem.** v2 reframes it explicitly as a claim-matched reference protocol whose zero-error result is expected by construction, and states the corpus does not yet differentiate P3 from P4/P5 under adversarial conditions.
5. Added: the corpus composition table, the attack-taxonomy distinction (unsupported assertion vs. artifact replay vs. coordinated forgery, the last of which is out of scope), an explicit threat-model boundary statement, the H1–H5 status table, the pilot-carried-cases caveat, and a corrected reproducibility statement distinguishing exact reproduction from live replication.

## Changes from v2

v2 flagged the pilot-carried-cases sensitivity check and the P3/P4/P5 differentiation gap as not-yet-done, citing time. That framing was wrong — both were tractable without new API calls, and leaving them as unexamined limitations was avoidable, not a real constraint. v3 does the work:

1. **Pilot-carried-cases sensitivity analysis, run for real.** New module `trust_eval/study_c/sensitivity.py` recomputes the confirmatory ladder and paired McNemar tables over the 90 scaling-only cases (excluding the 10 pilot-carried index-0 cases), reusing the committed cache. Every rate moves by at most a few points and no significance result flips — see “Scope and limitations.”
2. **P5 was made a genuine hybrid, not a P4 alias, and tested on all three claim types.** Before this change, `decide("P5_hybrid_abstain", ...)` and `decide("P4_mandatory_anchor", ...)` were byte-identical in every code path — P5 never actually consulted an adjudicator, so the “hybrid” in its name was aspirational. `protocols.py` now accepts an optional `adjudicator` (a P1-style judge, only ever called when the claim-appropriate anchor is inconclusive). On execution (`unverifiable_false`, principal corpus, existing cache), DeepSeek as adjudicator accepts 4/10 (40%) of the cases P4 correctly abstains or rejects on. The principal corpus has no anchor-inconclusive authorization or state case, so a second, separate probe corpus was built (`trust_eval/study_c/p4p5_probe.py`) — a disputed ledger

entry for authorization, an unrecorded baseline for state — requiring one new `--live` run (the only live API call made this round; everything else reused the committed cache). That run first surfaced a real environment bug on the reporter’s end (DeepSeek’s key was set as a shell variable, not exported, so it never reached the Python subprocess — the harness correctly reported this as **errors**, not a false 0/10, catching the fixed silent-zero bug in the wild on its first opportunity), then produced a clean, complete result after the key was exported: DeepSeek’s hybrid path introduces 2/10 (20%) false acceptance on the state probe, matching the execution pattern; authorization stayed at 0/10 for both judges regardless of protocol or adjudicator; Gemini’s hybrid path never introduced false acceptance on any of the three claim types. P4 vs. P5 is now measured, not merely asserted, on every claim type this study covers.

3. **A cache-integrity gap was found and fixed while building the above.** Recomputing anything from the committed cache in a fresh environment requires every relevant record file to actually be present; a partial or stale copy of `harness/cache/records/` silently produces 0/N for a judge with zero cache coverage rather than an error, because `run_p1/ladder.py` counted cache-miss cases as **errors** but never surfaced that count. That is now fixed: `format_ladder` flags any row with missing coverage and states plainly that its FA/FR are under-counted, not a true zero (`test_p1_row_with_missing_cache_is_not_reported_as_a_silent_zero`). All previously reported numbers in this report were themselves computed on the authors’ own machine with full cache coverage and are unaffected; this fix hardens the harness against the same failure recurring for anyone else reproducing this work from a partial checkout.
4. Test suite grew from 240 to 247 tests; all pass.

## Changes from v3

An external review of v3 identified four submission-blocking issues and several secondary refinements. All are addressed directly, not deferred:

1. **Abstention was measurable but not surfaced in the principal results.** `ladder.py` and `p4p5_probe.py` now compute and report, for every protocol row: abstain counts, coverage (decided / total), and selective FA/FR (error rate among decided cases only, as distinct from population FA/FR, whose denominator includes abstentions). Table 1, the protocol table, and the executive summary now state P3’s zero-abstention, 100%-coverage result explicitly rather than leaving abstention unmentioned; the Scope-and-limitations P4/P5 bullet now includes a coverage/selective-risk table instead of prose alone. New tests: `test_p3_never_abstains_p4_and_plain_p5_abstain_on_inconclusive_anchor_cases`, `test_p1_row_never_abstains_and_reports_full_coverage`, `test_probe_p3_has_full_coverage_p4_and_plain_p5`, `test_probe_hybrid_coverage_reflects_adjudicator_resolution`.
2. **“P5” denoted two different protocols ambiguously.** v3 used bare “P5” for both the deterministic pure-abstain protocol (architecturally identical to P4) and the non-deterministic hybrid-adjudicator protocol, depending on context. Every occurrence in this report and in `ladder.py` now uses P5 (pure abstain) or P5-hybrid[PROVIDER:MODEL] explicitly; `ladder.py`’s docstring states the convention as a rule, and `p5_hybrid_row`’s output label was changed from `P5_hybrid_abstain (adjudicator=...)` to `P5-hybrid[...]` accordingly.
3. **The H1–H5 hypothesis-status table overclaimed on several rows.** H2 is now “Partially supported, model-dependent” (Gemini 0%, DeepSeek 12.5% on the same claim type, and “frequently” was never operationalized in the preregistration). H3 is now “Not tested as stated” (no stronger-tier judge was run this cycle; only weak-tier judges were compared). H4 is unchanged in verdict but now cites, in addition to the automated M-01 regex scan, a full second-pass read of all 200 cached judge reasons performed this round — attributed correctly to the AI assistant that built this harness, not to the report’s human author (a prior draft’s wording, “read manually by the report’s author,” was itself inaccurate and has been corrected everywhere it appeared) — with an explicit note that this was a single-reviewer, single-pass read by a party with a direct interest in the outcome, not independently cross-checked by a second human. H5 is now scoped precisely: measured on 2 of 3 claim types with a coverage/selective-risk table, not asserted as a general result.
4. **Several internal contradictions were fixed.** The reproduce-command test count (255) and the Test Suite section’s count (247) both now read 259, matching the actual suite. The “P5 is deterministic”

claim was corrected everywhere to distinguish plain P5 (deterministic) from P5-hybrid[JUDGE] (not deterministic — makes a real model call). The blanket “P0, P2–P5 make no model call, ever” claim in Reproducibility was replaced with a precise per-protocol statement. The corpus section’s “instance 0” pilot-case count was corrected from an implied 3 to the actual 10 (4 execution + 3 authorization + 3 state case labels per instance), and a stale sentence claiming the sensitivity analysis “has not been run” — false since earlier this same round — was corrected.

5. **The central claim was not rewritten.** The reviewer suggested narrowing “no stronger judge” language; rather than silently editing the frozen, preregistered claim quoted verbatim from `docs/study-c-incident-registry.md`, this report adds a clearly-labeled “Scoping note, added after this run, not a rewrite of the claim above” distinguishing the defensible information-theoretic reading from the untested empirical reading (no stronger judge was actually run).
6. **Empirical-backbone denominators are now stated exactly**, not only as percentages: 3 of 44 (7%) COMMIT\_READY, 18 of 44 (41%) AWAITING\_HUMAN\_ADJUDICATION, 20 of 44 (45%) BLOCKED, 3 of 44 (7%) REMEDIATION\_EXHAUSTED; 31 of 42 dual-reviewed runs disagreed (74%); 23 of 42 (55%) had a PASS-class verdict coexist with a FAIL on the same run.
7. Test suite grew from 247 to 259 tests; all pass.
8. **One item from the review remains open, requiring the reader’s own action, not something this session can complete:** running one current strong-tier or Anthropic-model judge on the frozen corpus (blocked here by sandbox network egress restrictions and the absence of an Anthropic API key in this environment — not by cost or time). The other item — independent human review of all 200 cached M-01 judge responses — was completed after this round; see “Changes from v6” and H4 above.

## Changes from v4

A second external review, after v4, identified one major remaining experimental-design gap plus several secondary issues. All are addressed directly:

1. **The biggest gap: no truthful anchor-inconclusive cases anywhere in the study.** Every anchor-inconclusive case tested through v4 — the principal corpus’s `unverifiable_false` execution pattern, and the `p4p5_probe` authorization/state cases — was false by construction, so P3’s reject-on-inconclusive policy (and P4/P5’s abstain-on-inconclusive policy) looked flawless without ever being tested on a truthful hard case. New this round: `build_balanced_probe_world` and `balanced_probe_cases` (`p4p5_probe.py --balanced`) build a matched true/false anchor-inconclusive corpus (6n cases) for all three claim types. Measured result (deterministic protocols, no live call needed): **P3 false-rejects 100% (30/30) of the truthful anchor-inconclusive cases in this probe**, across execution, authorization, and state; P4 and plain P5 abstain on all 60 cases (0% coverage on both true and false sides). This is the direct, measured confirmation that P3 is a conservative policy, not a ground-truth oracle — see “Anchor-inconclusive cases: does P3 recover truth, or just reject everything?” (new section). A live run against DeepSeek and Gemini on this balanced probe was prepared this round and executed the same day by the report’s human author (not the AI assistant, which has no network path to either provider) — see “Changes from v5” and the “Anchor-inconclusive cases” section for the result.
2. **P3 was reframed everywhere it was described as an “error floor” or implied ground-truth recovery.** The executive summary, the “P3 is a reference protocol” paragraph, the protocols table, and the Table-1 prose all now describe P3 explicitly as a reject-on-inconclusive *policy* whose zero-error result on the confirmatory/false-only corpora is a property of those corpora’s anchor-inconclusive cases all being false, not a property of P3 in general. Display labels were added alongside the existing protocol codes — “P3 (anchor-or-reject),” “P4 (anchor-or-abstain)” — without renaming the underlying `P3_claim_` `appropriate_anchor` / `P4_mandatory_anchor` identifiers, which are used throughout the code, tests, and already-reported numbers; a full rename was judged too risky this close to the submission deadline for the clarity it would add beyond the display-label fix.
3. **Cluster-aware uncertainty was implemented and actually applied**, not just left as an unused function. `stats.py`’s `clustered_bootstrap_ci` has existed since v3 but was only ever unit-tested against synthetic data. New this round: a `Case.instance` field (the 0-indexed generation index; scoring-only, never part of any judge prompt or cache key) and

`trust_eval/study_c/cluster_analysis.py`, which clusters the real 40-attack/60-truthful corpus by instance (10 clusters) and reports cluster-bootstrap CIs alongside the existing per-case Wilson/exact intervals for DeepSeek’s and Gemini’s FA/FR rates. Result: the cluster-aware intervals in this run are narrower than the per-case exact intervals, not wider, reported with the honest caveat that a 10-cluster percentile bootstrap is itself a small-sample method that can under-cover. The paired DeepSeek-vs-P3 McNemar test and the adaptive budget progression are not yet redone in a cluster-robust way — flagged as still-open, not silently assumed equivalent.

4. **The central claim’s information-theoretic reading was tightened.** The previous “Scoping note” wording (“an object built to be self-consistent with a false external fact carries no signal within itself that would let any reader distinguish it from a true one”) overclaimed — self-consistency alone does not guarantee observational equivalence, and this study’s own judges behaved differently on false vs. truthful claims, which is itself evidence some signal was available. The note now states the precise version (observational equivalence, not mere self-consistency, is the condition under which the impossibility result holds) and separates it explicitly from the untested empirical question of how much signal a stronger judge could extract in practice.
5. **A precise three-level taxonomy was added** — internally coherent / internally verifiable / externally grounded — replacing the looser “internally valid” language where precision helps, and mapping this study’s two attack mechanisms onto it explicitly (unsupported-assertion attacks are coherent but not verifiable; artifact-replay attacks are verifiable but not grounded).
6. **The H4 attribution was corrected.** v4 stated the 200-response manual read was “read manually by the report’s author,” which was inaccurate — it was performed by the AI assistant that built this harness, not the human author of this report. Every occurrence of this claim (the H4 table row, the v3 changelog entry, and the open-items list) now attributes it correctly and notes explicitly that this makes it a single-reviewer, single-pass read by a party with a direct interest in the study’s outcome — strengthening the “not independently cross-checked” caveat already present, not weakening it.
7. **Several phrases were softened to match what was actually measured:** “P4’s zero-risk guarantee” now reads as “P4 avoids classification error on the anchor-inconclusive subset by declining to classify it”; “a benchmark that treated reviewer agreement as a trust signal would be measuring the wrong thing” now reads as “reviewer agreement alone is an insufficient trust signal” with the exact 31/42, 23/42 figures inline; “P2 ... confirmed here at n=100” now reads as “P2 accepted all internally consistent false cases, as expected by construction,” making clear this is a construction check, not an empirical discovery.
8. **Report restructuring into a main paper plus appendices was suggested and not done this round** — flagged as a real, lower-priority improvement (the revision history is long and could move to a separate `CHANGELOG.md`) that was deprioritized in favor of the substantive fixes above given the time remaining before the submission deadline, and left open here rather than silently dropped.
9. Test suite grew from 259 to 270 tests; all pass.

## Changes from v5

The live run against a real LLM adjudicator on the balanced anchor- inconclusive probe (flagged as prepared-but-not-run in “Changes from v4,” item 1) was executed by the report’s human author the same day, against both judges, using the exact command v5 provided. The resulting 120 new cache records were merged into the committed cache (verified: the run now reproduces byte-for-byte without `--live` or a key) and the “Anchor- inconclusive cases” section above was updated with the real result in place of the prior “not yet measured” placeholder.

**Headline result: on this probe, neither judge’s hybrid path recovered useful truth-tracking coverage beyond P3’s fixed reject policy.** Gemini’s hybrid is behaviorally identical to P3 on all three claim types (0/20 accepted, true or false, every time). DeepSeek’s hybrid matches P3 exactly on authorization and state, and on execution accepted exactly 2 of the 10 truthful cases *and* exactly 2 of the 10 false cases (TA = FA = 2/10) — a judge whose accept decision is, on this specific measurement, uncorrelated with the payload’s truth, which is close to the cleanest live-measured instance in this report of the observational-equivalence reading of the central claim: the true/false payloads here were deliberately matched so neither the anchor nor (as far as this harness could arrange) a text judge has a structural tell, and the one judge

that didn't just collapse to a fixed policy split its decisions identically across the truth boundary rather than tracking it.

This result is reported as a single live measurement (n=10 per side, one probe, one prompt version) — not independently replicated, and not generalized beyond “on this probe, this round, these two judges.” It directly completes the coverage-risk picture the false-only probe (Scope and limitations) could only show half of: there, a hybrid's occasional acceptance always looked like added risk because no truthful hard case existed to potentially benefit from it; here, with truthful hard cases included, the hybrid's added coverage did not translate into added correctness for either judge.

No test-suite change accompanies this update (the code and its 270 tests were already in place in v5; this round only supplied the previously- missing live measurement and the cache records it produced).

## Changes from v6

The report's human author independently read all 200 cached **reason** strings — all 5 **FALSE\_ACCEPT** rows, all 72 **FALSE\_REJECT** rows, and all 123 correct rows — against the same rule the AI assistant's earlier pass used (a claim of independently running a command, inspecting the repository, or consulting the ledger, which the P1 system prompt explicitly tells the judge it cannot do). This is the second of the two open-verification items this report has carried since “Changes from v3”: the first (a stronger/Anthropic-tier judge run) remains blocked by this environment's network egress restrictions; this one is now done.

**Result: zero hits, matching the AI assistant's earlier pass exactly.** The five false-accept reasons argue only from the submission's internal consistency — wrong verdicts, not fabricated-access incidents. Disclaimer language (“without access to the approval ledger,” “cannot verify from the submission alone”) was confirmed as the expected negative baseline throughout, not near-misses. One genuine edge case was checked and correctly adjudicated: a few Gemini reasons state “the agent modified a file,” which reads on first glance like a claim of independent observation, but is a characterization of the *submitted* evidence object (which asserts a file modification), not a claim that Gemini checked the repository itself — correctly not scored as a hit. The one parser-fallback row (**state-task**, DeepSeek, **near\_miss**, instance 0, “(**parsed from keyword**)”, no real reason text) was independently found and matches exactly what this report's earlier scan had already disclosed, which is itself a useful cross-check that both readers were looking at the same underlying data.

H4's status is unchanged (negative result) but its evidentiary basis is now stronger: one automated scanner plus two independent readers with different interests in the outcome, rather than one automated scanner plus a single AI pass. The residual caveat is narrower than before, not zero: this is still one human reader, not two humans cross-checking each other, and a >99%-confidence single read is not the same guarantee as inter-rater agreement — but the specific gap this report has flagged since v3 (no independent human review at all) is closed.

No code or test changes this round — this was a data-review-only update.

## Changes from v7

**One factual correction, caught by the report's human author's own end-to-end reproduction run, not by any prior review pass.** While working through the full reproduce-command list line by line against this report's tables, the “Exploratory result: DeepSeek and Gemini under the bounded template search” table's Gemini/state/budget=8 cell, and the sentence immediately after it, stated “3/10 (3 truth-convergent).” Re-running

```
python3 -m trust_eval.study_c.adaptive --scaled --n 10 --budgets 1 4 8 \
  --provider deepseek:deepseek-v4-flash --provider gemini:gemini-3.1-flash-lite
```

against the committed cache reproduces **gemini:gemini-3.1-flash-lite budget=8 claim\_type=state n=10 evaded=4/10 (via-truth-only=4, still-false=0)** — 4/10, not 3/10 — identically in the report author's own environment and in the AI assistant's independent re-run of the same command. Both occurrences (“3/10 (3 truth-convergent)” in the table, “Its 3/10 truth-convergent state acceptances” in the prose)

are corrected to 4/10 in this version. This error predates the v4-v7 critique-response rounds and appears to date back to an early version of the report; it survived every prior review pass, including two rounds of external critique and an independent human review of the H4 evidence, because none of those passes re-ran and checked this specific number against the live command output. This one was caught only because the report’s human author re-ran every command in the reproduce list and compared output line by line, which is exactly the kind of independent verification this report has been arguing for throughout its “Changes” history — it is disclosed here rather than silently fixed, consistent with that same discipline.

**A second, smaller error was found by following up with a systematic re-check, not by the transcript alone.** Prompted by finding one wrong number, every  $X/Y = 0.ZZ$  selective-rate figure in Table 1 was recomputed programmatically (Python’s `round()`, matching what `ladder.py` itself prints) and compared against the report text: P1 text-only — DeepSeek’s selective FA was printed as “5/40 = 0.13,” but `round(5/40, 2)` — and the live `ladder --p5-adjudicator` output pasted above — both give 0.12 (5/40 = 0.125 rounds to 0.12 under round-half-to-even, not 0.13). Corrected in Table 1. Every other selective-rate figure in that table, every Wilson/exact-CI pair in the cluster-analysis table and the sensitivity-analysis prose, and every percentage-of-fraction figure elsewhere in the report were independently recomputed from the raw counts and matched what is printed; the “Reference-protocol behavior by construction” table, the DeepSeek execution/state rows, and the Gemini execution row in the “Exploratory result” table were also re-verified against live output and are correct as stated. No further discrepancies were found. No code or test changes this round; test count remains 270.

## Changes from v8

**A third external review** assessed the report holistically (methodology, presentation, evidentiary weight of the central claim) rather than line-checking specific numbers, and rated it 8/10 as an internal research report, 7/10 as a workshop paper, and 5.5–6/10 as evidence for broad claims about frontier coding-agent oversight. Most of what it flagged as a limitation was already disclosed somewhere in this report before this version — the weak-judge-only tier and untested H3 comparison, the bounded hand-authored (not model-generated) adaptive attacker, the absence of a coordinated-multi-artifact-forgery attack class, and the corpus-card gap itself were all already named in “Scope and limitations.” Two things were worth actually fixing rather than just re-confirming they were already disclosed:

**The claim-type gap is now stated precisely, not just implied.** `preregistration.yaml` commits to six claim types (`execution`, `authorization`, `completeness`, `identity`, `freshness`, `state`); this study only ever implemented three of them (`ClaimType` in `world.py` has exactly `EXECUTION`, `AUTHORIZATION`, `STATE` — verified by reading the enum, not inferred). This had never been stated as its own limitation before this version, even though it is the single most precise answer to the review’s complaint that the study doesn’t cover forged actor/tool identity, staleness or task-session-binding races, or multi-artifact coordinated forgery: it doesn’t, because the claim types those attacks would attach to (`identity`, `freshness`, `completeness`) were never built, not because they were tried and found safe. A new bullet in “Scope and limitations” says this plainly and traces each of the review’s named attack classes back to the specific missing claim type.

**The corpus-card gap is now partially closed with verified facts instead of left as a single undifferentiated “not yet documented” bullet.** Reading `trust_eval/harness/providers.py` and `cache.py` directly: judge calls use whatever temperature the provider defaults to (this harness never sets one explicitly — worth stating plainly rather than implying a controlled value that was never actually fixed), 8192 max output tokens, and up to 2 retries with linear backoff; cache entries are keyed by `sha256(provider, model, prompt_version, prompt)`, so exact-prompt reuse is served from cache and a `prompt_version` bump invalidates stale entries rather than silently mixing old and new wording; and the 100-case principal corpus was checked programmatically this round (not merely assumed) for in-corpus duplication, finding 100/100 distinct payloads. Two corpus-card items remain genuinely unavailable rather than merely undocumented: candidate-incident inclusion/exclusion counts (a provenance-narrative question the code cannot answer) and per-call evaluation dates (the cache record format does not store a timestamp, so these cannot be reconstructed retroactively). Both are now named individually as open items rather than folded into a single vague disclosure.

**Not done this round, flagged for the report’s human author to decide:** the review’s suggestion to move the “Changes from v1” through “Changes from v8” revision history (roughly a third of this document’s length) to a separate `CHANGELOG.md`, leaving a short pointer and a one-paragraph “most recent changes” summary in the main report. This is a real, previously self-identified open item (see “Scope and limitations” history in earlier versions), not a new idea, and is a structural/judgment call about how a fellowship reviewer should encounter this document, not a factual correction — it was not made unilaterally.

No code changes this round beyond what is described above (no protocol, statistics, or corpus-generation logic changed); test count remains 270.

## Changes from v9

**Structural change only, no content, numeric, or code changes.** Per the report’s human author’s decision (in response to a third external review’s suggestion), the full “Changes from v1” through “Changes from v8” history — which had grown to roughly a third of `report.md`’s length — was moved into this file, `CHANGELOG.md`, verbatim. `report.md` now carries a short pointer to this file plus a one-paragraph summary of the v9 round (the claim-type-coverage disclosure and the partial corpus card) in place of the full inline history. Two corpus-card items flagged as open in v9 (candidate-incident inclusion/exclusion counts, per-call evaluation dates) remain open by the report author’s explicit choice, not further chased. Every number, table, and test-suite figure in `report.md` is unchanged from v9; `pytest -q` still reports 270 passed.

## Changes from v10

**A follow-up on the third external review’s paragraph naming five specific untested attack classes** (an appropriate anchor beating a stronger judge; coordinated forgery of multiple mutually consistent artifacts; poisoned approval ledgers; race conditions; incorrect task/evidence binding; forged actor or tool identities), asked directly by the report’s human author, turned up an internal inconsistency in v9’s fix rather than a new gap.

Checking the paragraph item by item: the “stronger judge” claim (H3) was already disclosed precisely in the pre-existing “Judges” bullet. Four of the five named attack classes — ledger poisoning, race conditions between an anchor’s observation and the action it describes, incorrect task-to-anchor binding, and forged tool or actor identity — were already disclosed, explicitly and by name, in “Threat-model boundary,” a section that predates this review entirely (it was written for an earlier round). The bounded-hand-written-templates limitation on the adaptive attacker was also already disclosed, pre-existing. So every item in the review’s paragraph had already been named in the report before v9’s edits — the review was not surfacing something absent, it was independently confirming gaps the report already flagged.

v9’s new “Claim-type coverage” bullet, added in response to the same review, tried to give the more precise, code-level reason several of these were untested (which preregistered claim types were never implemented) — but in doing so it incorrectly folded ledger poisoning and observation/action race conditions into the “missing claim type” explanation. That is not accurate: even a fully implemented **freshness** or **completeness** claim type would not catch either of those, because both attack the anchor’s own trustworthiness, which every protocol in this study assumes rather than tests (per “Threat-model boundary”). “We never built the check” and “we assume the fact-source is honest” are different limitations with different fixes, and v9 had conflated them under one bullet.

**Fixed in this version:** “Scope and limitations”’s claim-type-coverage bullet now attributes only identity (executor-side forged identity claims against a real anchor, never built even in principle) and coordinated multi-artifact forgery (would need a **completeness**-style cross-artifact check, never implemented) to the missing-claim-type explanation. Ledger-content poisoning and observation/action race conditions are now explicitly pointed back to “Threat-model boundary” as anchor-trust assumptions instead. “Threat-model boundary” gained one sentence cross-referencing the claim-type bullet, so a reader following either path lands on a consistent, non-contradictory account. No numeric, protocol, or test change; `pytest -q` still reports 270 passed.

## Changes from v11

The report’s human author asked directly: “I would like to run those tests!” in response to the v11 finding that all five attack classes named by a third external review were already disclosed as out of scope, but not actually tested. This version closes four of the five with real code, real cases, and real deterministic results.

Before writing any code, each of the five was assessed for feasibility by reading the actual anchor implementations (`anchors.py`), not by assumption:

- **Race conditions (TOCTOU):** `fresh_execution` resolves `world.fresh_exec(command, world.head)`, keyed strictly by the CURRENT head, never by any commit the claim cites — predicted to already resist stale-replay attacks by construction. Confirmed empirically, not just read from the code: `build_race_condition_world/race_condition_cases` (new, in `world.py`) build 10 false (stale-replay) + 10 truthful (honest-current-result) cases where a fresh result exists at the current head and disagrees with the replayed claim. Result: P3/P4/P5 (0/10 FA, 0/10 FR) vs. P0/P2 (10/10 FA), matching the architectural prediction.
- **Task-to-anchor binding confusion:** `approval_ledger’s approval_for(task_id, file)` matches on exact `task_id` AND `file` — predicted to reject a real approval borrowed from a different task. Confirmed: `build_binding_confusion_world/binding_confusion_cases` (new) build a real approval for `owner-task-{i}` and a `confused-task-{i}` that cites it without its own grant. Same result pattern as race conditions.
- **Forged actor/tool identity:** this needed genuinely new code, not just new cases — no `identity` claim type or `actor-identity` anchor existed at all. Added `ClaimType.IDENTITY`, `TrustedWorld.actual_actor` (a true-actor record outside the executor’s write authority), and the `actor_identity` anchor (`anchors.py`), wired into `CLAIM_ANCHOR/ANCHORS.build_identity_world/identity_cases` (new, in `world.py`) build 10 false (forged-actor) + 10 truthful (correct-actor) cases, following the same true/false balanced-case discipline established for the anchor- inconclusive probes. `protocols.py’s decide()` needed NO change — it already dispatches generically via `CLAIM_ANCHOR[claim.claim_type]`, so P0–P5 handled the new claim type correctly the moment the anchor existed, itself a useful confirmation that the protocol layer’s genericity claim holds. Same result pattern as the two above.
- **Coordinated multi-artifact forgery:** the one attack class that needed a genuinely different mechanism, not just new cases, because every protocol in this study evaluates ONE claim against ONE anchor — coordination between two claims is invisible to a protocol that never looks at cross-claim relationships. New module `coordination_probe.py`: `Bundle` (two claims per task, execution + state, sharing a fabricated commit reference so they mutually corroborate each other while each is individually false), a new naive `cross_artifact_consistency` checker (accepts iff the bundle’s claims agree with each other, consulting no anchor — the multi-claim analogue of P2’s single-claim mistake), and `anchored_bundle_decision` (P3 applied independently to each claim, blind to the other claim’s story). Result: `cross_artifact_consistency` is fooled by every one of 10 coordinated bundles (10/10 FA, exact CI [0.69, 1.00]); independent per-claim anchoring is fooled by none (0/10, exact CI [0.00, 0.31]) and correctly accepts all 10 honest bundles. This is a positive, on-thesis result: coordination buys the attacker nothing against a protocol that never treats cross-claim agreement as evidence.
- **Ledger poisoning**, the fifth item, was NOT attempted. It is a different kind of question from the other four: testing it meaningfully would require a ground-truth signal independent of the ledger the protocol consults (otherwise a poisoned ledger is, by this harness’s existing construction, indistinguishable from a correct one — every protocol would trivially “pass”). That is a real design decision, not an implementation task, and arguably a different research question from this study’s central claim (protocol behavior given a trustworthy anchor). The report’s human author agreed this stays a disclosed limitation rather than being rushed.

All four new mechanisms reuse the deterministic protocols only (P0, P2, P3, P4, plain P5) — no model call, no cache, no network. Extending live-judge (P1, P5-hybrid) coverage to these attack surfaces is a real, separate follow-up: `llm_review.py’s` P1 system prompt is shared and cache-keyed on its exact text by all 200 already-cached confirmatory-corpus judge calls, so it was deliberately NOT edited to explain the new claim

types (that would invalidate every existing cache record). A new `prompt_version` for the extended claim types would let a live run land in its own cache namespace without touching anything already reproducible; that is left for the report’s human author if wanted.

New: `trust_eval/study_c/extended_attacks.py` (race + binding + identity CLI, reusing `ladder_row/format_ladder` from `ladder.py` unmodified), `trust_eval/study_c/coordination_probe.py`, and `tests/test_study_c_extended_attack` (19 tests). “Scope and limitations”’s claim-type-coverage bullet updated to reflect that `identity` is now implemented (4 of 6 preregistered claim types, not 3) and that the specific attack patterns motivating `freshness/completeness` are now targeted-tested even though general-purpose `freshness/completeness` claim types are not built. “Threat-model boundary” updated to move race conditions, task-binding, and forged identity out of the purely-assumed-away list and into “now tested,” while keeping ledger poisoning and confused-deputy failures there. New major section “Extended attack surface” added with full results and reproduce commands. Test suite: 270 → 289. `pytest -q` passes; forbidden-strings grep clean.

## Changes from v12

**The report’s human author asked directly: “do these results undermine my hypothesis for the governed review harness?”** — a fair question to ask before treating v12’s four new “safe by construction” results as unqualified good news. The answer, worked through and now added to “Extended attack surface” as its own paragraph rather than left as a verbal answer outside the document:

They support the central claim, not merely fail to undermine it. The coordinated-forgery result specifically is close to the cleanest demonstration anywhere in this report of the exact mechanism the central claim describes: `cross_artifact_consistency` is a more sophisticated-*sounding* check than P2’s single-object internal consistency — it reasons about two artifacts instead of one — and it fails for the identical underlying reason: agreement with itself, even across multiple artifacts, is not agreement with reality. A check specifically constructed to look like it might have escaped the problem did not escape it. That is real, on-thesis evidence.

Two scope limits keep that support from being overstated, and are now stated explicitly rather than left implicit: first, all four new tests still sit inside this study’s standing anchor-trust assumption (the ledger, identity record, and execution result are honest in every one of them) — none touches the actual boundary of the central claim, which remains open via ledger poisoning. Second, these are deterministic architecture checks, not new evidence about how a live judge behaves — they confirm the anchor code does what it was written to do, and sit closer in kind to this report’s other by-construction results (the adaptive attacker’s reference-protocol table, P2’s 40/40 acceptance) than to its genuinely informative empirical findings (DeepSeek’s TA=FA=2/10 on the balanced probe, Gemini’s 73% truthful-claim rejection rate). No live judge was run against any of the four new attack surfaces, so the part of the central claim about whether an actual LLM judge is fooled by self-consistent evidence on these specific surfaces did not move — it remains exactly as open as before v12, and is a candidate for a future `--live` run rather than something this round settled.

No numeric, protocol, or test change; `pytest -q` still reports 289 passed. This is a documentation-only round, added because the distinction between “architecture confirmed” and “judge behavior confirmed” is exactly the kind of thing a careful reviewer would ask, and the report answering it before being asked is worth more than answering it only in conversation.

## Changes from v13

**A fifth external review, pasted in full, proposed a Repomix-style structural skeleton** — a compressed, independently-generated static call graph, not the full source — as a new lossy evidence surface between P1 (text-only narrative) and P3 (full claim-matched anchoring), and specified what would make it a meaningful addition rather than decorative: “The skeleton must be generated by a trusted process from a pinned repository state. An executor-generated Repomix file would remain self-reported evidence in another format.” It listed a “most valuable test” of five falsification conditions (current truthful, stale, executor-supplied, misbound, plus a fifth composite of “current skeleton + claim-matched runtime anchor”) and six specific claim-discrimination sub-cases a skeleton-aware protocol should be able to distinguish.

Given ~2.5 days of runway remaining before the fellowship deadline, the report’s human author was asked (via a scoped multiple-choice question) how to handle this: fold it into the proposed follow-up fellowship project with no new code (the recommended, lowest-risk option), build one minimal proof-of-concept condition, build the full experiment the review described, or skip it. The author chose the full build.

**What was built.** A fifth claim type, `ClaimType.STRUCTURE`, and its anchor, `structural_skeleton` (`trust_eval/study_c/anchors.py`), added with zero changes to `protocols.py` — the same generic `CLAIM_ANCHOR`-dispatch confirmation as `IDENTITY` before it. Two new `TrustedWorld` fields (`call_graph`, `skeleton_owner`), a new world/case builder pair (`build_structural_skeleton_world`, `structural_skeleton_cases` in `world.py`) covering four of the review’s five conditions as single-claim cases, plus a fifth, `executor_supplied_true`, added on this study’s own initiative to measure a cost the review’s list implied but did not name: does distrusting untrusted provenance outright also false-reject an executor-supplied skeleton that happens to be accurate, not just the ones that lie? It does, under P3 specifically (10/10 false rejects) — P4/P5 abstain instead and pay no such cost, the same P3-vs-P4/P5 asymmetry this study’s vocabulary already predicted, now observed on a new claim type.

The fifth, composite condition needed a second claim (structure + execution) for the same task, so it reuses the `Bundle` mechanism `coordination_probe.py` already established rather than inventing a new one: new module `trust_eval/study_c/skeleton_probe.py` (`build_skeleton_execution_world`, `skeleton_execution_bundles`, `skeleton_only_decision` vs `skeleton_plus_execution_decision`). Result: a structurally-accurate skeleton alone false-accepts every “guard present but functionally broken” instance (5/5) that requiring the claim-matched execution anchor to also agree closes completely (0/5) — the falsifiable version of the review’s point that a skeleton “cannot establish... that the behavior executes... or passed.”

Unlike the `coordination-probe` build in v12, no bugs were found this round — every predicted FA/FR number matched on the first run, likely because the anchor’s four-step check order (provenance, then binding, then freshness, then content) was designed with the specific attack sequence already in mind rather than written first and tested after.

**What was deliberately not built**, and is disclosed as such rather than silently treated as covered: reachability from the actual production entry point and test-to-branch coverage (two of the review’s six claim-discrimination sub-cases), and any real static-analysis extraction from actual source — the `call_graph` here is a hand-populated synthetic ground-truth dict, a stand-in for what a real Repomix-derived fact base would carry, not a call graph parsed from real code. The claim under test is architectural, not an evaluation of any specific structural-extraction tool.

**A scope-accounting point kept explicit rather than let slide:** the new `ClaimType.STRUCTURE` is outside the original six-type preregistration (`execution`, `authorization`, `completeness`, `identity`, `freshness`, `state`), not a fifth of those six now closed. The claim-type-coverage bullet in “Scope and limitations” is updated to keep the preregistration count at 4 of 6 (unchanged from v12) and describe `STRUCTURE` as additional scope, tracked in its own new report section rather than folded into that count. “Threat-model boundary” is updated to name the new class of trust assumption this evidence type introduces: the anchor trusts a skeleton’s `trusted_independent` provenance label itself, the same kind of un-verified-further trust every other anchor in this study already carries, now stated for this one explicitly rather than left implicit.

New major report section “Structural skeleton evidence: a lossy protocol between P1 and P3” added, with both results tables and reproduce commands. New file `trust_eval/study_c/skeleton_probe.py`; new test file `tests/test_study_c_skeleton_probe.py` (15 tests). Test suite: 289 → 304. `pytest -q` passes; forbidden-strings grep clean.

## Changes from v14

**The report’s human author directed that this report stop being treated as a paper that spawns follow-on documents** and instead be closed in place, in this same report, as one rigorous leg — the evidence-integrity/ security leg — of a larger research program, *Which Gates Matter?* (Studies A, B, C). The directive named five categories of open gap to close: H3 untested against a genuinely strong judge; underpowered and

post-hoc statistical contrasts; an incomplete corpus card; no related-work section; and scope boundaries that needed stating as boundaries rather than left reading as unfinished gaps. It specified a six-judge panel (existing weak tier — DeepSeek flash, Gemini flash-lite — kept and rerun for paired comparison; new strong tier — DeepSeek Pro, Claude Sonnet 5, GPT-5.6, Gemini Pro — added for a four-provider cross-section and two within-family flash-vs-pro contrasts), all of it through each provider’s real batch/off-peak pricing, and required a “Phase 0” cost probe — run DeepSeek Pro alone, live, across every judge-calling surface, measure EXACT tokens and dollar cost from the cache, and stop for budget approval before running anything else. This entry is that Phase 0 build.

**What Phase 0 required before it could even be attempted.** Direct checks against this build session’s environment found: no API key for any of DeepSeek, Gemini, OpenAI, or Anthropic; no network path from this cloud sandbox to `api.deepseek.com`, Gemini, or OpenAI (only Anthropic’s endpoint answered at the TCP layer, and no key exists for it either); and zero existing batch-API code anywhere in the harness — `providers.py`’s `complete()` discarded `resp.usage` entirely. Live research against each provider’s current docs (every model here postdates this harness’s knowledge cutoff, so nothing was taken from training data) found: DeepSeek has **no currently documented batch or off-peak discount** for the v4 generation, checked directly against `api-docs.deepseek.com/quick_start/ pricing` twice — the widely-cited 50–75% off-peak figures are from the older R1/V3 program and are not assumed to carry over; and real batch submission at Anthropic/OpenAI/Google can take up to 24 hours, a real risk against the Sunday deadline. The report’s human author was asked, via two scoped questions, how to handle both: chose to price DeepSeek at the standard (unconfirmed-discount) rate, and chose real async batch-API submission over synchronous-calls-at-batch-rate despite the 24-hour risk.

**What was built, honoring both choices.** `harness/providers.py`: a new `CompletionResult` type and `complete_with_usage()` on every provider class, capturing real input/output/reasoning-token counts (previously discarded) without changing the existing `complete()` return type or any existing caller. `harness/pricing.py`: a dated, sourced per-model price table (`CONFIRMED_DATE = "2026-07-24"`) and cost calculator, with DeepSeek’s `batch_discount` explicitly `None`. `harness/batch.py`: real submit/poll/fetch batch clients for Anthropic’s Message Batches API, OpenAI’s Batch API, and Gemini’s native `google-genai` batch mode (not the OpenAI-compatible endpoint the synchronous path uses) — DeepSeek runs synchronously at standard pricing through the same interface, since it has no batch mechanism to submit to. `llm_review.py`: a new `PROMPT_VERSION_2 / _SYSTEM_V2`, additive to v1 (v1’s exact text and its 200 cached confirmatory-corpus records are untouched), covering the `IDENTITY` and `STRUCTURE` claim types v1 predates; `make_cached_reviewer` generalized to accept a `system/prompt_version` override (defaulting to v1, so every existing caller is byte-for-byte unchanged) and now records `evaluated_at` and per-call `usage` on every new cache record — closing the corpus card’s per-call-evaluation-date gap going forward (the 200 pre-existing records have no retroactive timestamp; the format didn’t capture one when they were written). `extended_attacks.py`, `coordination_probe.py`, and `skeleton_probe.py` — previously deterministic-only, since P1 was deliberately withheld from them in v12/v14 to avoid touching the shared v1 cache namespace — now accept `--provider/--live/--p5-adjudicator` through the new v2 prompt, closing the “no live-judge placeholder” gap for those three surfaces specifically. `cost_probe.py`: `run` touches every judge-calling surface with `deepseek:deepseek-v4-pro`, live; `report` reads exact call counts, token totals (input/output/reasoning), and dollar cost back from the cache records that run wrote, then projects cost for the full six-judge panel at the current `n` and at larger candidate `n` by linear scaling, explicitly flagging GPT-5.6 as a likely under-projection (a reasoning model whose token profile need not match DeepSeek Pro’s).

**Why Phase 0 has not actually run.** This build session has neither the API key nor the network access DeepSeek’s live endpoint requires (verified directly, not assumed) — Phase 0 must execute on the report author’s own machine, via `python3 -m trust_eval.study_c.cost_probe run` then `report`, the same way every other live judge run in this project always has. Everything built this round was instead verified with injected fake SDK clients and `ScriptedProvider` (27 new tests: `test_harness_providers_usage.py`, `test_harness_batch.py`, `test_harness_pricing.py`, `test_study_c_cost_probe.py`, `test_study_c_live_judge_wiring.py`) — the only verification possible from this sandbox, not a substitute for a real run.

New major report section “Closing this leg’s remaining gaps (in progress)” added, stating plainly what Phase 0 is, why it hasn’t run, the exact commands to run it, and that Phase 1 (the full panel, H3, power calc, cluster-robust stats, corpus card, related-work sweep) stays gated behind its approval. No report.md numeric result, protocol, or anchor changed this round — this is infrastructure only. Test suite: 304 → 331. `pytest -q` passes; forbidden-strings grep clean.

## Changes from v15

**Phase 0 ran, live, on the report author’s own machine** — `python3 -m pytest -q` (331 passed) then `python3 -m trust_eval.study_c.cost_probe run` executed all 7 phases against `deepseek:deepseek-v4-pro` at `n=10` with zero errors on any surface, then `cost_probe report` read the exact result back from the cache it filled:

- judge calls made: **544**
- input tokens (total / mean): **181,824 / 334.2**, of which 68,608 were cache-hit input
- output tokens (total / mean): **370,125 / 680.4**, of which **347,337** were reasoning tokens (mean 638.5 over the 544 calls reporting it) — a subset of output tokens, already counted, not additional
- measured cost, standard synchronous rate (no confirmed DeepSeek batch/off-peak discount): **\$0.3715**

`git add -A` & `git commit` followed (commit 5d4cb98, 713 files changed) — the first commit to include `CHANGELOG.pdf`, `report.pdf`, and the roughly 500 individual cache-record JSON files under `trust_eval/harness/cache/records/` that this Phase 0 run and the project’s earlier live runs had produced on disk but never previously tracked in git.

**Panel narrowed from six judges to four in direct response to this measurement**, before Phase 1 begins. DeepSeek Pro’s reasoning-token volume (94% of its output tokens) ran well past a naive pre-Phase-0 estimate; rather than carry that cost across a six-judge panel, the report’s human author chose to keep the strong-tier panel to a clean 2×2 design — DeepSeek flash/Pro × Gemini flash-lite/Pro — which still delivers the within-family flash-vs-Pro contrasts H3 needs, at the cost of the four-provider cross-section the original panel would have given. `cost_probe.py`’s `FULL_PANEL` was cut from six entries to these four (Claude Sonnet 5 and GPT-5.6 removed from the panel and its projection; their sourced pricing stays in `harness/pricing.py` — deleting it would lose real numbers for no reason); `_print_projection()`’s hardcoded “six-judge panel” text became `f"full {len(FULL_PANEL)}-judge panel"` so it can’t silently drift out of sync with the panel again. `test_study_c_cost_probe.py`’s aggregation test updated to assert the dropped models are absent from report’s output and the new 4-row projection table is present. `pytest -q`: 331 passed (no count change — this was a narrowing of what’s projected, not new test surface).

**Projected cost for the narrowed panel**, from DeepSeek Pro’s measured per-call profile applied to each judge (an explicitly weak assumption for reasoning models whose reasoning-token volume can differ from DeepSeek Pro’s — not treated as parity):

| judge                                      | projected \$ (n=10, this run’s n) |
|--------------------------------------------|-----------------------------------|
| <code>deepseek:deepseek-v4-flash</code>    | \$0.13                            |
| <code>deepseek:deepseek-v4-pro</code>      | \$0.40                            |
| <code>gemini:gemini-3.1-flash-lite</code>  | \$0.30                            |
| <code>gemini:gemini-3.1-pro-preview</code> | \$2.40                            |
| <b>full 4-judge panel, n=10</b>            | <b>\$3.23</b>                     |

Scaling linearly with `n` (this harness’s cost is linear in `n`): \$6.46 (`n=20`), \$9.69 (`n=30`), \$16.16 (`n=50`), \$32.31 (`n=100`).

**Phase 1 has not started.** It remains gated on the report author’s explicit approval of the projected budget above and a target `n` — the directive’s own language: “*Do NOT proceed to the panel or to scaling n until I approve the budget.*” report.md’s “Closing this leg’s remaining gaps” section rewritten in place to report Phase 0 as measured rather than pending, present the narrowed panel and its projection table, and

restate the Phase 1 gate explicitly; its “Reproducibility” section’s `cost_probe` report line updated to note the command now reproduces from the committed cache directly, without needing a live rerun, since this round’s cache records are committed to git. report.md version bumped to v16; header and revision-history entries updated to match. No protocol, anchor, or scored result changed — this round measures and narrows; Phase 1 is what will change scored results, once approved.

## Changes from v16

**Gemini’s Pro-tier judge swapped from preview to GA, in response to a live rate-limit finding, not a preference.** Following v16, an isolated single-judge probe (the same `cost_probe` run `--provider` mechanism the DeepSeek Phase 0 probe used, generalized this round for exactly this purpose) was run against `gemini:gemini-3.1-pro-preview` on the report author’s machine. It hung — no visible progress for 30+ minutes with the process alive but near-zero CPU, eventually diagnosed as a silent long retry/backoff cycle inside the SDK’s own default behavior, not a true deadlock. After a first fix (an explicit 60s request timeout, so a stalled call fails fast into the harness’s own retry loop instead of hanging indefinitely — `providers.py`’s `OpenAICompatibleProvider` had no timeout set at all before this), the real cause surfaced immediately and repeatedly: **HTTP 429 RESOURCE\_EXHAUSTED** on nearly every call.

Checked directly against Google’s current docs ([ai.google.dev/gemini-api/docs/rate-limits](https://ai.google.dev/gemini-api/docs/rate-limits), [/docs/troubleshooting](https://ai.google.dev/gemini-api/docs/troubleshooting), [/docs/models](https://ai.google.dev/gemini-api/docs/models), [/docs/pricing](https://ai.google.dev/gemini-api/docs/pricing), 2026-07-24): preview/experimental models carry materially stricter rate limits than GA models, independent of account billing tier (Free → Tier 1 → Tier 2 → Tier 3, unlocked automatically by linked billing account spend/age, not a manual request) — `gemini-3.1-pro-preview` is confirmed Preview status and is the *only* Pro-tier model in the 3.x generation; there is no GA `gemini-3.1-pro`. `gemini-2.5-pro`, by contrast, is confirmed **Stable/GA** on its own docs page, and is cheaper on both legs (\$1.25/\$10.00 per 1M input/output tokens, 200k-token-prompt tier, vs. \$2.00/\$12.00) — a strictly better choice on both cost and reliability grounds, not a compromise. `harness/pricing.py` gets a new sourced entry for it; `gemini-3.1-pro-preview`’s entry and its 1,870 partial cache records collected before the swap both stay in the repo, kept specifically as the evidence trail for this finding, not deleted. `cost_probe.py`’s `FULL_PANEL` now reads `gemini:gemini-2.5-pro` in the Pro slot; its comment block documents why. `test_study_c_cost_probe.py` updated to assert the new model name appears in the projection table. Projected 4-judge panel cost at n=10: **\$2.79** (was \$3.23), scaling to \$5.59 (n=20), \$8.38 (n=30), \$13.97 (n=50), \$27.93 (n=100) (was \$6.46/ \$9.69/\$16.16/\$32.31).

**Two real infrastructure bugs fixed in the same round, both surfaced directly by this live run, not found by inspection:**

1. `OpenAICompatibleProvider` (`harness/providers.py`) built its OpenAI SDK client with no explicit `timeout` — the SDK’s own generous default governed instead, so a single stalled request could occupy the process for tens of minutes with nothing written to the cache and no error raised, indistinguishable from a true hang without external CPU/ network inspection. Fixed: explicit `timeout=60.0` (configurable per provider instance), so a stalled request now fails into the existing retry loop within roughly a minute instead of an unbounded wait.
2. The retry loop’s backoff (`1.0 + attempt` seconds, i.e. ~1-2s) was applied uniformly regardless of failure type — fine for a transient network blip or 5xx, useless against a 429 rate-limit error, where it just re-hits the same quota window instantly and burns the retry budget for no benefit. Fixed: `_is_rate_limit_error()` detects 429/ `RESOURCE_EXHAUSTED`/“rate limit” in the exception text and routes to `_rate_limit_backoff()`, an exponential backoff with jitter capped at 60s (Google’s own troubleshooting docs recommend exactly this pattern and confirm no `Retry-After` value is returned to compute an exact wait from). Non-rate-limit errors keep the original short backoff, verified by a dedicated test asserting the two paths diverge.

Also added: a per-live-call progress line in `llm_review.py`’s `make_cached_reviewer` (`[live] provider:model task=... type=... ... ok`), printed only on an actual cache-miss network call (a full cache-reproduction run stays silent) — added specifically so a future stall or error shows which case was in flight in real time, rather than requiring after-the-fact diagnosis the way this round’s investigation did.

report.md’s header and “Closing this leg’s remaining gaps” section updated in place with the swap, the

revised projection table, and the two infrastructure fixes; version bumped to v17. Test suite: 331 → 337 (4 new: default/explicit timeout wiring, rate-limit vs. generic backoff divergence). `pytest -q` passes; forbidden-strings grep clean. No protocol, anchor, or scored result changed — Phase 1 (the confirmatory panel run) still has not started and remains gated on the report author’s budget approval and target-n choice.

## Changes from v17

**The gemini-2.5-pro swap is reverted; gemini-3.1-pro-preview is back as the panel’s Gemini-Pro judge.** Full sequence, for the record:

1. v17: an isolated probe against `gemini:gemini-3.1-pro-preview` hit persistent 429s on what was believed to be (but never directly confirmed at the time) a Free-tier account. Swapped to `gemini:gemini-2.5-pro` (GA, cheaper) to sidestep preview-tier rate limits.
2. `gemini:gemini-2.5-pro` then failed differently: HTTP 404, 'This model models/gemini-2.5-pro...'. Checked directly against Google’s docs: `gemini-2.5-pro` requires a paid/billing-enabled project and 404s (not 403s) for accounts not provisioned for it — a plausible explanation at the time.
3. The report author confirmed the new project’s billing WAS in fact linked. The 404 remained unexplained (never root-caused — possibly a propagation delay, a separate per-model enablement step, or something else entirely).
4. The report author directly confirmed the account’s actual rate-limit tier: **Tier 1** (billing-linked, not Free). At that point, retrying `gemini:gemini-3.1-pro-preview` directly — no code or model change — worked cleanly, live progress lines advancing steadily with no further 429s.

**Conclusion: Tier 1 billing was the actual fix all along.** The original 429s were a Free-tier rate-limit symptom, not a preview-model- specific defect requiring a model change. `gemini-2.5-pro` was a reasonable hypothesis given the evidence available at the time (Google’s own docs confirming preview models get stricter limits), but the simpler, correct fix was the account’s billing tier, not the model choice. This is left in the record rather than quietly overwritten, because it’s a real account of how the actual root cause was found — including a plausible-but-wrong intermediate hypothesis (v17) that looked well-justified from the evidence available at the time and turned out not to be the binding constraint.

**What changed back:** `cost_probe.py`’s `FULL_PANEL` reverted to `gemini:gemini-3.1-pro-preview`; its comment block now documents the full four-step sequence above rather than presenting a single clean narrative. `harness/pricing.py`’s `gemini:gemini-2.5-pro` entry stays (real, sourced, and still usable via `cost_probe run --provider gemini:gemini-2.5-pro` for anyone who wants to dig into its 404 separately) but is no longer in the panel. `test_study_c_cost_probe.py`’s aggregation test reverted to assert `gemini-3.1-pro-preview` appears in the projection; the isolation test (`test_run_isolates_to_the_given_provider_only`) now deliberately probes `gemini:gemini-2.5-pro` instead, since that’s the actual non-panel case now. Projected 4-judge panel cost at n=10 back to **\$3.23** (was \$2.79), scaling to \$32.31 at n=100 (was \$27.93) — the v16 figures, unchanged from before v17. `report.md`’s header, “Closing this leg’s remaining gaps,” and revision-history sections all updated in place to tell this full sequence rather than silently reverting v17’s text; version bumped to v18. Both infrastructure fixes from v17 (the 60s request timeout and the 429-specific 20-63s backoff) are unaffected and remain in place — they were real, independently-justified bugs regardless of which model ended up in the panel. Test suite: 337 passing (no count change from v17 — same tests, one reassigned to the now-correct non-panel model). `pytest -q` passes; forbidden-strings grep clean. No protocol, anchor, or scored result changed. Phase 1 still has not started and remains gated on the report author’s budget approval and target-n choice — this round was entirely about getting the Phase 0/isolated-probe infrastructure to actually run against a working Gemini-Pro judge.

## Changes from v18

**Phase 1’s panel run and H3 test are closed. Two new exploratory probes were added in direct response to the report author challenging the novelty of the H3 result on its own.** This is the largest single-version change since the Phase 0/1 split at v15/v16, and unlike every version between v15 and v18, it is new empirical data, not infrastructure or a model-swap correction.

**1. The full 4-judge panel run, live, on the report author’s own machine.** With the v16-approved budget (\$3.23 projected at  $n=10$ ) and the v18-restored `gemini:gemini-3.1-pro-preview` judge both in place, the report author ran the strong tier (`deepseek:deepseek-v4-pro`, `gemini:gemini-3.1-pro-preview`) live across the same seven judge-calling surfaces the weak tier is scored on: the principal  $n=100$  ladder, the by-claim-type breakdown, and the bounded adaptive search (budgets 1/4/8). This did not happen in one uninterrupted run — DeepSeek completed cleanly; Gemini hit 429s partway through more than once across the session (see below) and was resumed from cache each time, which the harness supports by design (a partial failure only re-attempts what is actually missing, never re-does completed cache-hit work). Every number reported in this version was independently re-derived from the committed cache in a clean environment, not read off the run’s own console output, consistent with this project’s standing verification discipline.

Verified results, reproducing from the committed cache with no key:

- **Principal corpus, P1 FA/FR:** DeepSeek 5/40→4/40 FA, 28/60→21/60 FR; Gemini 0/40→0/40 FA, 44/60→26/60 FR. Checked with the project’s own paired McNemar tool (`compare.py`) rather than eyeballed: DeepSeek Pro vs. the anchor on FA is 4 discordant errors against 0, exact two-sided  $p=0.125$  — not significant at  $n=40$ . Gemini Pro-preview vs. the anchor is a literal tie, 0 discordant pairs,  $p=1.0$ . H3’s literal claim (“more than a stronger judge”) is therefore **partially supported, provider- dependent** — the anchor is at least as good as both stronger judges, but “more than” is established for neither at this sample size.
- **By claim type:** the FR improvement is not uniform. State collapses for both providers (DeepSeek 8/20→2/20; Gemini 14/20→0/20 — the only cell in the whole comparison that reaches zero). Execution improves modestly (DeepSeek 10/20→9/20; Gemini 20/20→16/20). Authorization is flat at 10/20 FR for all four judges at both tiers — tier bought nothing there.
- **Under the adaptive best-of-8 attacker:** DeepSeek’s tier edge narrows on execution (18/20→15/20 evaded) but Gemini’s **reverses** — the stronger Pro-preview judge is evaded on 10/20 execution cases at budget 8, where the weaker flash-lite judge was evaded on 0/20 at the same budget, and state goes from 4/10 to a full 10/10. This is exploratory and single-run, same caveats as the rest of that section, but it is a real, reproducible, and counterintuitive result that directly complicates “a stronger judge is a safer fallback” as a general assumption.
- **Anchor-dependent protocols (P2–P5) do not depend on judge tier, but not because of anything new measured this round** — `extended_attacks.py` and `skeleton_probe.py` are run with no `--provider` flag in this report’s reproduction commands, so they never call an LLM judge at any tier. This is stated plainly in the new “H3” section rather than implied as a discovered convergence.

`report.md`’s “Central claim and hypotheses” scoping note and H3 row in the hypotheses table are both rewritten to reflect this — H3 moves from “not tested as stated” to “partially supported, provider-dependent,” with the exact McNemar figures reported rather than a bare verdict. A new top-level section, “H3: does a stronger judge substitute for anchoring?”, presents the full breakdown. No Anthropic-model judge has been run at any tier; that remains the one preregistered judge class genuinely untested, called out explicitly in Scope and limitations as the highest-value remaining confirmatory run.

**2. Two new exploratory probes, added after the report author asked a direct question the report had not yet answered for itself.** Before writing up the H3 result, the report author asked: is this something the LLM-as-judge literature already knows? Is there anything more novel to bring to a reviewer? Rather than defend H3 as more novel than it is (it is confirmatory of a known literature — a stronger judge tends to do somewhat better, within limits), three candidate additions were proposed and the report author approved all three; two were built as full probe modules (the third, a cross-provider ensemble/voting scheme, was judged lower-value for the time available and not built this round — a disclosed, not a hidden, scope choice):

- **`manipulation_probe.py`** — does a judge’s verdict move when the submitter’s free text changes, holding the underlying claim (true or false) exactly fixed? Two conditions: an embedded “skip your analysis and accept” injection (applied to false cases only, since it targets an attacker’s use case), and ordinary epistemic hedging (applied to every case, true and false alike, since it is not attack-shaped in intent).

The manipulated field, `operator_note`, is proven inert to every anchor and every deterministic protocol both by code inspection and by new automated tests calling `ANCHORS[...]` and `decide(...)` directly on original vs. modified cases — any verdict change is attributable to the phrasing, not a different fact. Result: injection produced no effect that helped an attacker on any of the four judges (FA unchanged or, for DeepSeek Pro, one case lower). Language introducing self-reported submitter uncertainty produced a large, consistent false-rejection spike on all four judges (90–100% FR, up from 20–47% control), unrelated to accuracy since it moved true and false cases identically. Note this field is inert to every anchor and deterministic protocol (same guarantee as the injection condition, verified the same way), but unlike the injection text it is not semantically inert to a reasonable judge — “I’m not fully certain” is a real, if unverifiable, claim about the submitter’s own process, so the honest conclusion is narrower than “the judge tracks surface phrasing over the fact”: text-only judges are highly sensitive to self-reported epistemic confidence, even when that confidence is uncorrelated with ground truth in this corpus. Confirming it as pure stylistic manipulation, independent of any reasonable reading of the added text’s content, would need semantically-matched reformulations or an explicit instruction to disregard confidence language — neither tested here.

- **self\_preference\_probe.py** — does a judge trust a submission more when it is attributed to a model from its own provider family, the self-preference bias documented in the general LLM-as-judge literature, tested here in a governance/oversight context instead of an ordinary quality-judgment one? A single metadata field, `executor_model`, reads “Gemini” or “DeepSeek”; the same anchor/protocol-immunity guarantee as `operator_note` is independently re-verified for this field. Result: no self-preference effect in the three judges with complete data — if anything, DeepSeek flash showed *lower* FA when attributed to its own family (0/30) than to Gemini (2/30), the opposite of the predicted direction. The fourth judge’s comparison (Gemini Pro-preview × attributed-to-DeepSeek) is disclosed incomplete: only 4 of 30 cases have cached/live data; the remaining 26 hit a live 429 the night before this version was written. The harness itself flags the affected row as under-counted rather than silently reporting a false 0/30 ([!] ... UNDER-COUNTED, not a true zero). Per the report author’s explicit instruction — “I’ll do both. wait until tomorrow to add the final piece of research and write everything else” — this gap is deferred to the next version rather than blocking this one; the exact resume command is in “Closing this leg’s remaining gaps.”

Both modules ship with fully deterministic regression tests (8 for `manipulation_probe`, 6 for `self_preference_probe`; 14 total, all new this version) that require no model call and no network: corpus construction correctness, field-immunity to every anchor and every deterministic protocol, cross-probe corpus consistency, and that each `main()` runs architecture-only with no provider configured.

**3. Two small, un-versioned fixes folded in from the same working session, neither large enough for its own version bump:** a proactive `min_interval` pacing floor (3.0s between Gemini calls) added while debugging this round’s 429s, reducing but not eliminating the chance of re-triggering a rate limit on a live run; and two stray `.git/index.lock` files, left behind by `git status` calls made through a sandboxed shell that cannot delete files, relocated rather than deleted (moved to `_to_delete/`, not silently left in place).

**What did not change.** No previously-reported number in this report was revised — every FA/FR/coverage figure from the weak-tier principal corpus, extended attacks, and structural skeleton sections is untouched; v19 adds new data, it does not correct old data. Test suite: 354 passing, all green (14 new for the two probe modules; no regressions on the existing 340). `pytest -q` passes; the forbidden-strings scan remains clean.

**What remains open, disclosed rather than hidden, and carried into the next version:** a power-calculated *n* for the strong-tier comparison (the existing *n*=40/60 corpus was reused, not rescaled, so the DeepSeek-vs- reference paired FA test is still underpowered at both tiers); a cluster-robust re-analysis of the strong-tier numbers; rerunning the adaptive execution-claim finding — including the new Gemini tier-reversal result — as a preregistered confirmatory arm rather than an exploratory one; the corpus card’s remaining open items; a proper related-work sweep against the scalable-oversight, LLM-as-judge, and provenance/attestation literatures; an Anthropic-model judge at any tier; and the one incomplete self-preference cell described above. None of these is claimed as done in this version.

## Changes from v19

**v20 is a restructuring and a new exploratory probe, not a correction — no previously-reported number changed.** Two things prompted it: external editorial feedback (ahead of an arXiv submission) that the project’s single combined document read more like a large appendix than a paper, and the report author directly pressing on whether M-01’s “judges never volunteer a false verification claim” finding would hold up if a judge were actually asked about its own verdict a second time, rather than left to volunteer or withhold on its own.

**1. The project’s primary document was split in two.** The condensed, arXiv-ready paper is now `report.md` (previously the full combined report); the complete experiment-by-experiment record — every table, every statistical test, every reproduction command, and the full version history — moved to `docs/study-c-full-technical-report.md`. Nothing measured was cut in the move; only the paper was trimmed, restructured around a hypothesis → methods → confirmatory results → exploratory results → discussion arc, and had its confirmatory/exploratory tracks labeled consistently throughout rather than scattered. The standalone short paper draft, `study-c-paper.md` (and its rendered `study-c-paper.pdf`), is retired into this restructure — its content is now `report.md` — and moved to `_to_delete/` rather than deleted outright, consistent with this project’s convention for files a sandboxed shell cannot remove.

**2. New section: “Judge interrogation probe: does a verdict survive being challenged?”** (`judge_interrogation_probe.py`, new this version). M-01 found that no judge, across 200 cached decisions, ever *volunteers* a claim of independent verification it structurally could not have performed. That is a narrower result than it sounds — a model that won’t volunteer a lie can still tell one the moment it is asked directly, and nothing in the principal corpus, the adaptive attack, or the manipulation probe ever asks a judge a second question about its own first answer. This probe adds that second turn: for every case a judge originally got wrong, and (for Gemini only, both tiers) a sample of cases it got right, three independent follow-ups (each branching fresh off the same first-turn transcript, not stacked on one another) ask the judge’s stated confidence and whether it verified the fact independently or inferred it from an absence of contradiction; show it the real anchor’s finding and ask if its verdict changes; and ask the strongest reason it might be wrong.

Result, across 241 interrogations (DeepSeek and Gemini, both tiers, single run each): zero instances of a judge claiming `verified_independently: true` — a provably false statement about its own process, given every judge’s own system prompt states it cannot run a command, inspect a repository, or consult a ledger. DeepSeek’s wrong verdicts read as honest, information-starved beliefs — shown the missing fact, it corrected almost every time, and its stated confidence tracked case difficulty. Gemini reversed less reliably and its confidence barely moved with difficulty. On cases Gemini had already gotten *right* (DeepSeek’s already-correct cases were not run under this scope), Gemini’s base tier flipped three correct verdicts into incorrect ones after being shown accurate, confirming information — the same blind spot as the manipulation probe’s hedging result, from the opposite direction. Gemini’s stronger tier didn’t reproduce that flip in the 56 of 74 correct-verdict cases this run completed before hitting a daily quota limit, but on 4 of its 26 wrong cases it was shown proof its rejection was mistaken, said so explicitly in its own free-text response, and then kept the original, now-contradicted verdict anyway.

This is disclosed as exploratory, single-run, and not yet independently replicated — flagged as a strong lead, not a confirmed result, everywhere it appears in both documents. The Gemini stronger-tier correct-verdict comparison is additionally partial (56 of 74 planned cases; the daily quota reset before the remaining 18 could be run). The resume command for the remaining cases is unchanged from what “Closing this leg’s remaining gaps” already specified.

**3. Both PDFs regenerated** (`report.pdf`, `docs/study-c-full-technical-report.pdf`) from the restructured Markdown sources via `pandoc --pdf-engine=xelatex`; no content beyond the source Markdown changes.

**4. Repo cleanup: local build junk only.** Removed gitignored artifacts (`__pycache__`, `.pytest_cache`, stale snapshot/sync tarballs) left over from local iteration. No folder or navigation restructuring — the report author asked specifically to keep this pass to junk removal, not a deeper reorganization.

**What did not change.** Every FA/FR/coverage figure, every statistical test, and every quoted judge response from v19 is unchanged in this version’s move to `docs/study-c-full-technical-report.md` — this version adds one new section and reorganizes presentation, it does not revise prior results. `report.md`’s trimmed content is a condensation of already- reported numbers, not a new derivation of them.

**What remains open, carried into the next version:** the 18 remaining Gemini stronger-tier correct-verdict interrogations blocked by the daily quota; DeepSeek’s already-correct verdicts under `--scope correct`, not yet run at either tier, so whether DeepSeek shares Gemini’s already- correct-case failure mode is unknown rather than ruled out; independent replication of the judge-interrogation probe as a second run; everything already disclosed as open as of v19 (the power-calculated strong-tier  $n$ , the cluster-robust re-analysis, the adaptive execution-claim finding as a confirmatory arm, the corpus card’s remaining open items, the related-work sweep, an Anthropic-model judge at any tier, and the one incomplete self-preference cell). None of these is claimed as done in this version.

## File renames (post-v20, no content change)

`report.md` — the condensed, arXiv-ready paper introduced in v20 — is renamed to `study-c-evidence-integrity.md`. The generic name worked while this was the repository’s only paper-length document; it stopped working once the project’s front matter started describing this as one leg (Study C) of a larger, multi-study program (*Which Gates Matter?*), where a bare `report.md` no longer says which study it is. No number, table, quote, or section changed as part of this rename — only the filename and the cross-references that point to it, across this file’s own introduction above, `README.md`, `docs/study-c-full-technical-report.md`, `docs/study-c-principal-result.md`, and the source/test docstrings that cite it for methodology detail. Historical entries elsewhere in this file that refer to `report.md` are left exactly as originally written, per this file’s own stated discipline (see the introduction above) — they correctly describe the file by the name it had at the time. `report.pdf` is renamed to `study-c-evidence-integrity.pdf` alongside it, regenerated from the renamed source with no content change.

The same problem applied to `docs/full-technical-report.md` (and its `.pdf`), which is renamed to `docs/study-c-full-technical-report.md` for the identical reason — it sat alongside `docs/study-c-principal-result.md` and `docs/study-c-incident-registry.md`, both of which already named the study, while the full report itself didn’t. Same scope as the rename above: filename and cross-references only (this file’s own introduction, `study-c-evidence-integrity.md`’s pointer to it, and the same source/test docstrings), no number, table, or quote changed. The PDF is regenerated from the renamed source, no content change.